Department of Artificial Intelligence and Machine Learning
B.Tech. Sem: V Subject: Full Stack Development Using NextJs (DJS23AMD302L)

EXPERIMENT NO. 4

AIM:

To create standard API Route Handlers (route.ts) alongside native React Server Actions to manage full CRUD data mutations (GET, POST, PUT, DELETE) without traditional API fetch boilerplates.

SOFTWARE REQUIRED:

- **Operating System:** Linux / macOS / Windows 10/11
- **Node.js runtime environment:** (v18.17.0 or higher)
- **Package Manager:** npm or pnpm
- **Code Editor:** Visual Studio Code

THEORY:

- **Next.js App Router Route Handlers (**route.ts**):** Route Handlers allow developers to create custom request handlers for a given route using the Web Request and Response APIs. They replace the legacy pages/api endpoints and support standard HTTP verbs (GET, POST, PUT, DELETE, PATCH). Route Handlers execute exclusively on the server, ensuring secure database access and external API secret management.
- **React Server Actions (**'use server'**):** Server Actions are asynchronous JavaScript functions that execute on the server. They can be called directly from Client or Server Components, or invoked inside HTML <form> action attributes. By leveraging Server Actions, developers can perform mutations directly from the UI layer without manually writing client-side fetch() requests, useEffect hooks, or managing manual loading state boilerplates.
- **Route Cache Invalidation (**revalidatePath**):** When mutations occur on the server (via Server Actions or Route Handlers), Next.js's aggressive client-side Data Cache and Router Cache may serve stale data. The revalidatePath('/route') function purges the cached data for a specific route path, forcing a silent server re-render of the component tree and serving fresh data seamlessly.

STEPS / ALGORITHM:

Step 1: Create an In-Memory Data Store (app/lib/db.ts**)**

Establish a simple shared state array to simulate a backend database for CRUD operations.

```
export interface BlogItem {
  id: string;
  title: string;
  content: string;
}

// In-memory data store for demonstration
```

```
export const postsDb: BlogItem[] = [
  { id: '1', title: 'Getting Started with Next.js 16',
content: 'Exploring App Router features.' },
  { id: '2', title: 'Mastering Server Actions', content:
'Simplifying data mutations without fetch.' },
];
```

Step 2: Implement RESTful API Route Handler (app/api/posts/route.ts**)**
Create standard HTTP endpoint handlers (GET, POST, PUT, DELETE) using Web Response objects.

```
import { NextResponse } from 'next/server';
import { postsDb } from '@/app/lib/db';

// GET: Fetch all posts
export async function GET() {
  return NextResponse.json({ success: true, data: postsDb },
{ status: 200 });
}

// POST: Add a new post via API
export async function POST(request: Request) {
  const body = await request.json();
  const newPost = {
    id: Date.now().toString(),
    title: body.title || 'Untitled Post',
    content: body.content || 'No content provided.',
  };
  postsDb.push(newPost);
  return NextResponse.json({ success: true, data: newPost },
{ status: 201 });
}

// PUT: Update an existing post
export async function PUT(request: Request) {
  const body = await request.json();
  const index = postsDb.findIndex((post) => post.id ===
body.id);

  if (index === -1) {
```

```
    return NextResponse.json({ success: false, message: 'Post
not found' }, { status: 404 });
  }

  postsDb[index] = { ...postsDb[index], title: body.title,
content: body.content };
  return NextResponse.json({ success: true, data:
postsDb[index] }, { status: 200 });
}

// DELETE: Remove a post
export async function DELETE(request: Request) {
  const { searchParams } = new URL(request.url);
  const id = searchParams.get('id');

  const index = postsDb.findIndex((post) => post.id === id);
  if (index === -1) {
    return NextResponse.json({ success: false, message: 'Post
not found' }, { status: 404 });
  }

  postsDb.splice(index, 1);
  return NextResponse.json({ success: true, message: 'Post
deleted' }, { status: 200 });
}
```

Step 3: Implement Native React Server Actions (app/actions/postActions.ts**)**
Define asynchronous server functions to handle form submissions and direct mutations without writing manual fetch calls.

```
'use server';

import { revalidatePath } from 'next/cache';
import { postsDb } from '@/app/lib/db';

// Server Action: Create Post
export async function createPostAction(formData: FormData) {
  const title = formData.get('title') as string;
  const content = formData.get('content') as string;
```

```
  if (!title || !content) return;

  postsDb.push({
    id: Date.now().toString(),
    title,
    content,
  });

  // Purge cache to instantly refresh UI data
  revalidatePath('/posts-demo');
}

// Server Action: Delete Post
export async function deletePostAction(id: string) {
  const index = postsDb.findIndex((post) => post.id === id);
  if (index !== -1) {
    postsDb.splice(index, 1);
  }

  // Purge cache to instantly refresh UI data
  revalidatePath('/posts-demo');
}
```

Step 4: Construct Fullstack UI View (app/posts-demo/page.tsx**)**
Create a page component that demonstrates Server Action mutations directly inside form components and standard client-side API interaction.

```
import { postsDb } from '@/app/lib/db';
import { createPostAction, deletePostAction } from
'@/app/actions/postActions';

export default function PostsDemoPage() {
  return (
    <div style={{ padding: '20px', maxWidth: '600px', margin:
'0 auto', fontFamily: 'sans-serif' }}>
      <h2>Experiment 4: Route Handlers & Server Actions</h2>

      {/* Form using native React Server Action (Zero Fetch
Boilerplate) */}
```

```
      <div style={{ border: '1px solid #333', padding:
'15px', borderRadius: '8px', marginBottom: '20px' }}>
        <h3>Add Post via Server Action</h3>
        <form action={createPostAction} style={{ display:
'flex', flexDirection: 'column', gap: '10px' }}>
          <input
            type="text"
            name="title"
            placeholder="Post Title"
            required
            style={{ padding: '8px', borderRadius: '4px',
border: '1px solid #555' }}
          />
          <textarea
            name="content"
            placeholder="Post Content"
            required
            style={{ padding: '8px', borderRadius: '4px',
border: '1px solid #555' }}
          />
          <button type="submit" style={{ padding: '10px',
background: '#2563eb', color: '#fff', border: 'none',
borderRadius: '4px', cursor: 'pointer' }}>
            Submit via Server Action
          </button>
        </form>
      </div>

      {/* Rendered Post List */}
      <h3>Posts List (Direct DB / State Reading)</h3>
      <div style={{ display: 'flex', flexDirection: 'column',
gap: '10px' }}>
        {postsDb.map((post) => (
          <div key={post.id} style={{ border: '1px solid
#444', padding: '10px', borderRadius: '6px', display: 'flex',
justifyContent: 'space-between', alignItems: 'center' }}>
            <div>
              <h4 style={{ margin: '0 0 5px 0'
}}>{post.title}</h4>
              <p style={{ margin: 0, color: '#aaa', fontSize:
'14px' }}>{post.content}</p>
```

```
            </div>

            {/* Inline Server Action triggering deletion */}
            <form action={deletePostAction.bind(null,
post.id)}>
              <button type="submit" style={{ background:
'#dc2626', color: '#fff', border: 'none', padding: '6px
12px', borderRadius: '4px', cursor: 'pointer' }}>
                Delete
              </button>
            </form>
          </div>
        ))}
      </div>
    </div>
  );
}
```

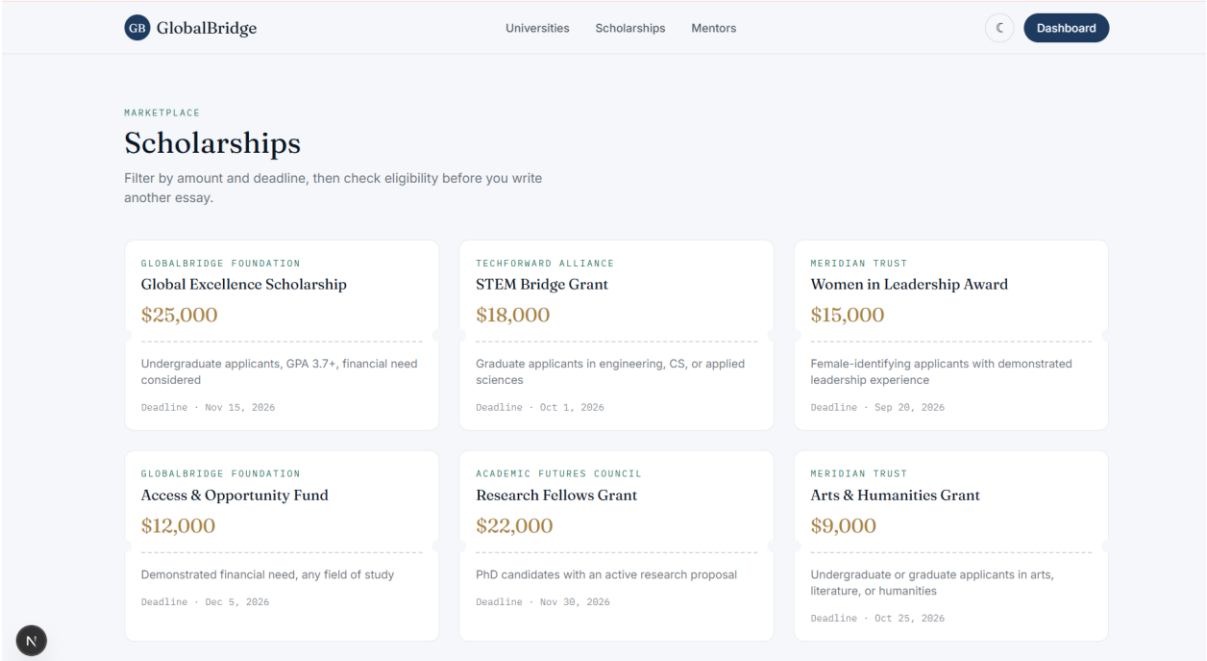
Step 5: Test and Validate Data Mutations

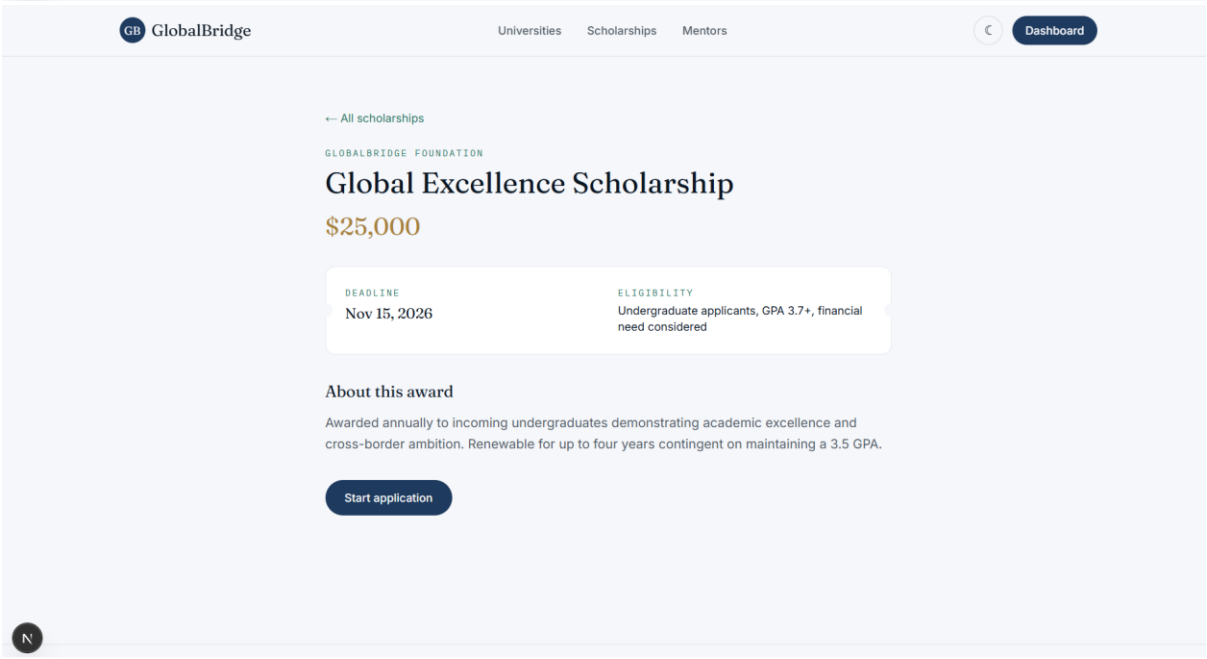
1. Execute npm run dev in the terminal.
2. Open http://localhost:3000/posts-demo in your browser.
3. Fill out the form and click **"Submit via Server Action"**. Observe that the page updates instantly without manual state management or fetch() code.
4. Test HTTP API endpoints via cURL or Postman:
    - **GET All:** GET http://localhost:3000/api/posts
    - **POST Item:** POST http://localhost:3000/api/posts with JSON body {"title": "API Post", "content": "Created via Route Handler"}

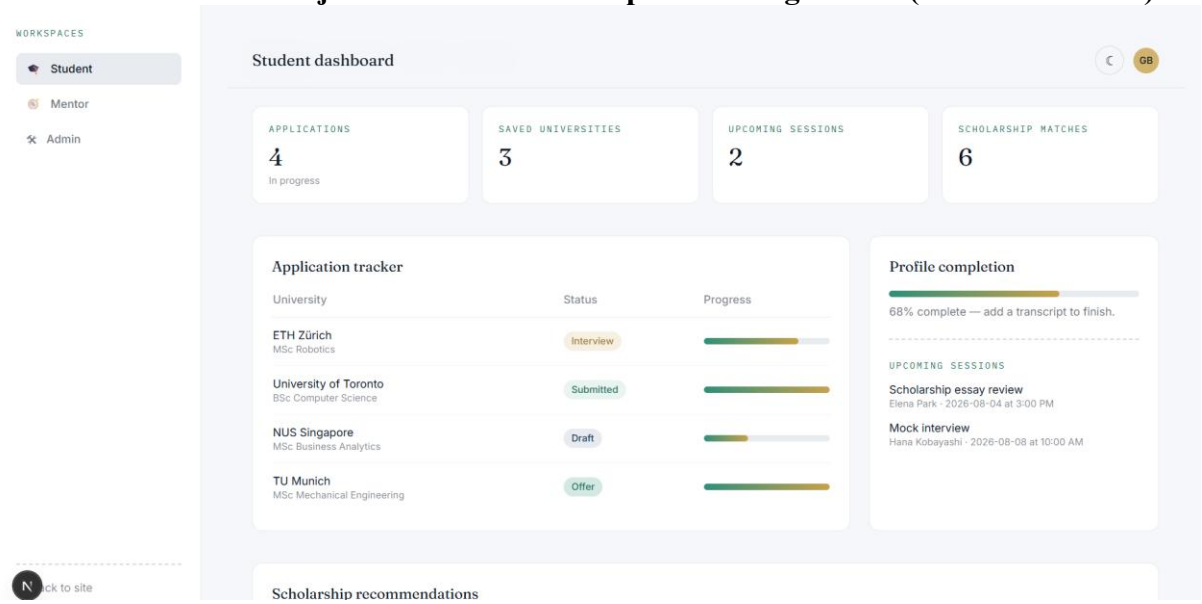
OBSERVATION:

Observe how submitting the form triggers the createPostAction function directly on the server without issuing a manual fetch('/api/posts') request. revalidatePath('/posts-demo') invalidates the server cache, triggering an automatic UI refresh with updated data.

Additionally, note how /api/posts/route.ts remains accessible for traditional REST integration (e.g., external mobile apps, third-party clients) alongside Server Actions.


GlobalBridge
Universities
Scholarships
Mentors
Dashboard
MARKETPLACE
Scholarships
Filter by amount and deadline, then check eligibility before you write another essay.
GLOBALBRIDGE FOUNDATION
Global Excellence Scholarship
$25,000
Undergraduate applicants, GPA 3.7+, financial need considered
Deadline · Nov 15, 2026
TECHFORWARD ALLIANCE
STEM Bridge Grant
$18,000
Graduate applicants in engineering, CS, or applied sciences
Deadline · Oct 1, 2026
MERIDIAN TRUST
Women in Leadership Award
$15,000
Female-identifying applicants with demonstrated leadership experience
Deadline · Sep 20, 2026
GLOBALBRIDGE FOUNDATION
Access & Opportunity Fund
$12,000
Demonstrated financial need, any field of study
Deadline · Dec 5, 2026
ACADEMIC FUTURES COUNCIL
Research Fellows Grant
$22,000
PhD candidates with an active research proposal
Deadline · Nov 30, 2026
MERIDIAN TRUST
Arts & Humanities Grant
$9,000
Undergraduate or graduate applicants in arts, literature, or humanities
Deadline · Oct 25, 2026


GlobalBridge
Universities
Scholarships
Mentors
Dashboard
← All scholarships
GLOBALBRIDGE FOUNDATION
Global Excellence Scholarship
$25,000
DEADLINE
Nov 15, 2026
ELIGIBILITY
Undergraduate applicants, GPA 3.7+, financial need considered
About this award
Awarded annually to incoming undergraduates demonstrating academic excellence and cross-border ambition. Renewable for up to four years contingent on maintaining a 3.5 GPA.
Start application



CONCLUSION:

Standard API Route Handlers (route.ts) and native React Server Actions were successfully constructed. The experiment verified full HTTP CRUD operations, eliminated traditional client-side API fetch boilerplates via Server Actions, and demonstrated automatic cache revalidation using revalidatePath.

POST LAB QUESTION:

Q: What are the security and performance benefits of using Server Actions over client-side fetch() calls to Route Handlers?

A:

1. **Reduced Bundle Size & Zero Boilerplate:** Server Actions eliminate the need to ship client-side fetching libraries, manual useState loading/error boilerplate code, and useEffect hooks to the browser, significantly reducing the JavaScript bundle size.

2. **Security & Server-Only Execution:** Server Actions execute exclusively on the server. Database credentials, secret keys, and complex validation logic are never exposed to the client bundle or network payload.

3. **Progressive Enhancement:** Forms backed by Server Actions can function even if JavaScript is slow to load or disabled on the client browser, whereas standard fetch() callbacks depend entirely on client-side JavaScript execution.